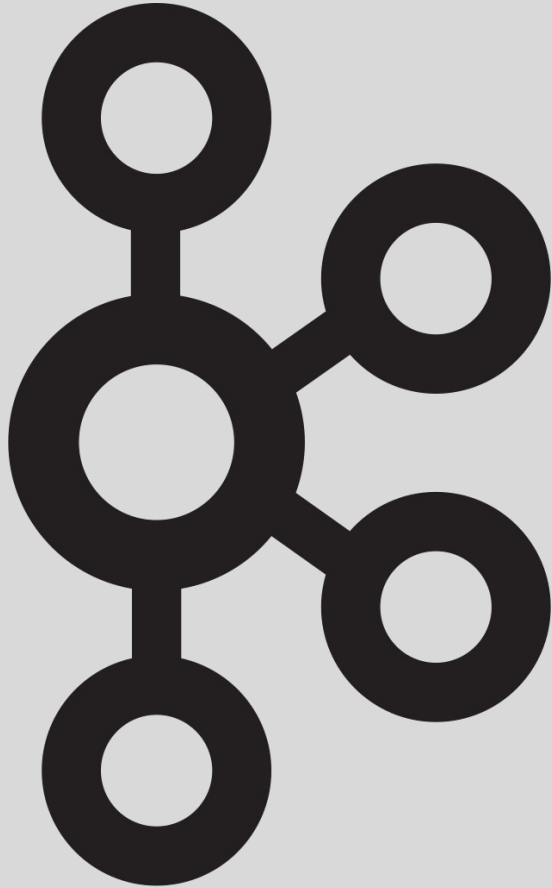


Kafka

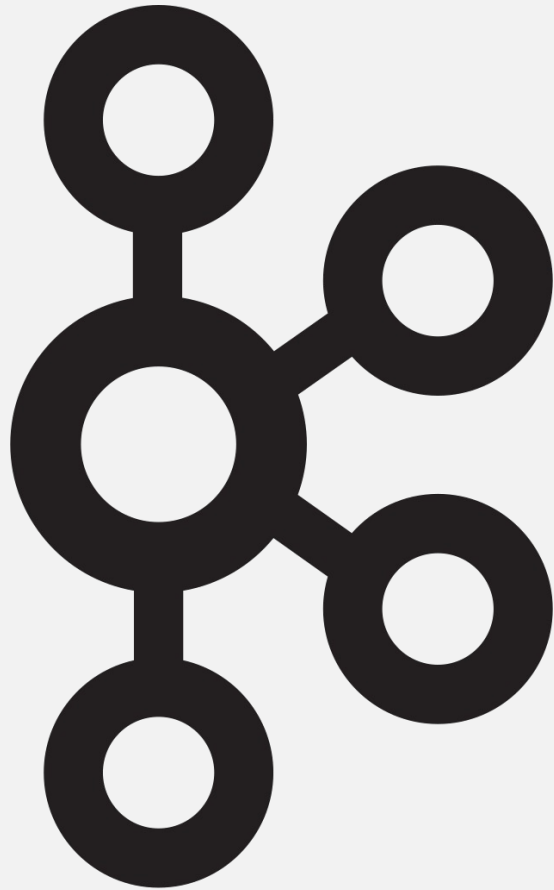
Streaming Ingestion



Agenda

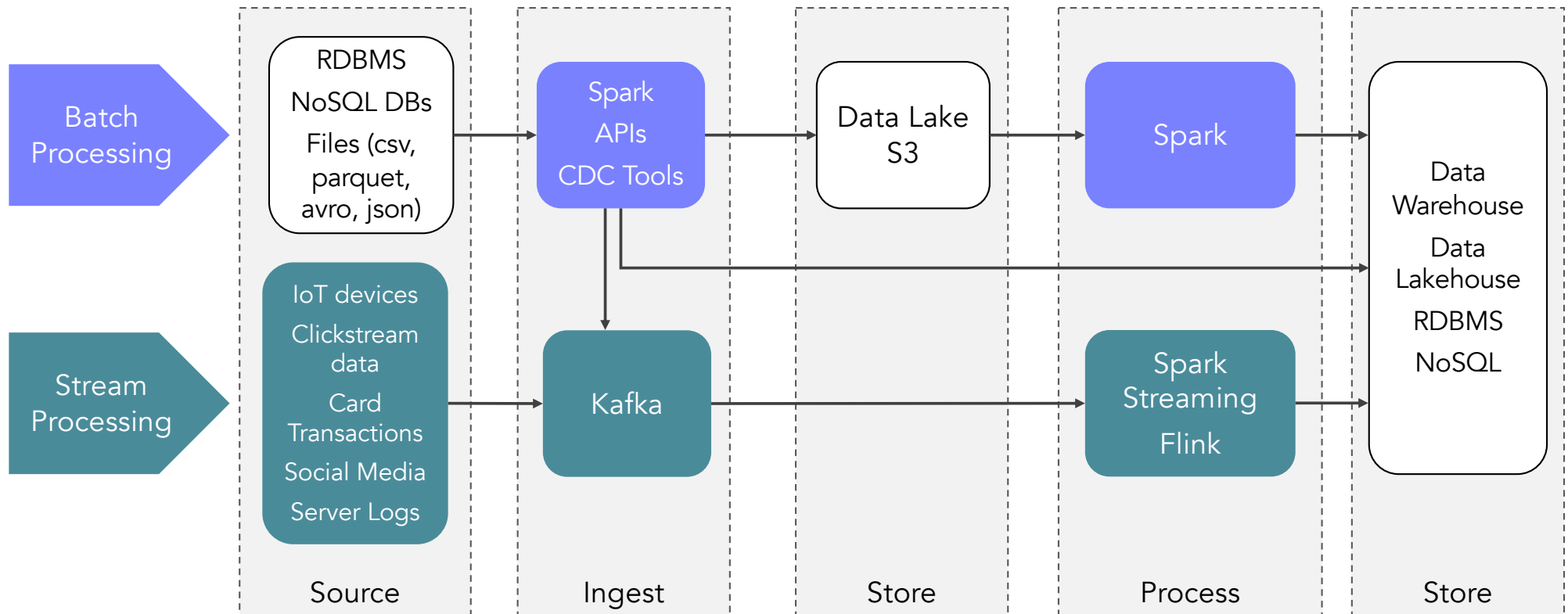
- Stream Processing
- Kafka Introduction
- Kafka Cluster and Components
- Knowing Kafka Up Close
- Producer Application
- Consumer Application
- Offset Handling
- How Spark uses Kafka

www.3aayaam.in



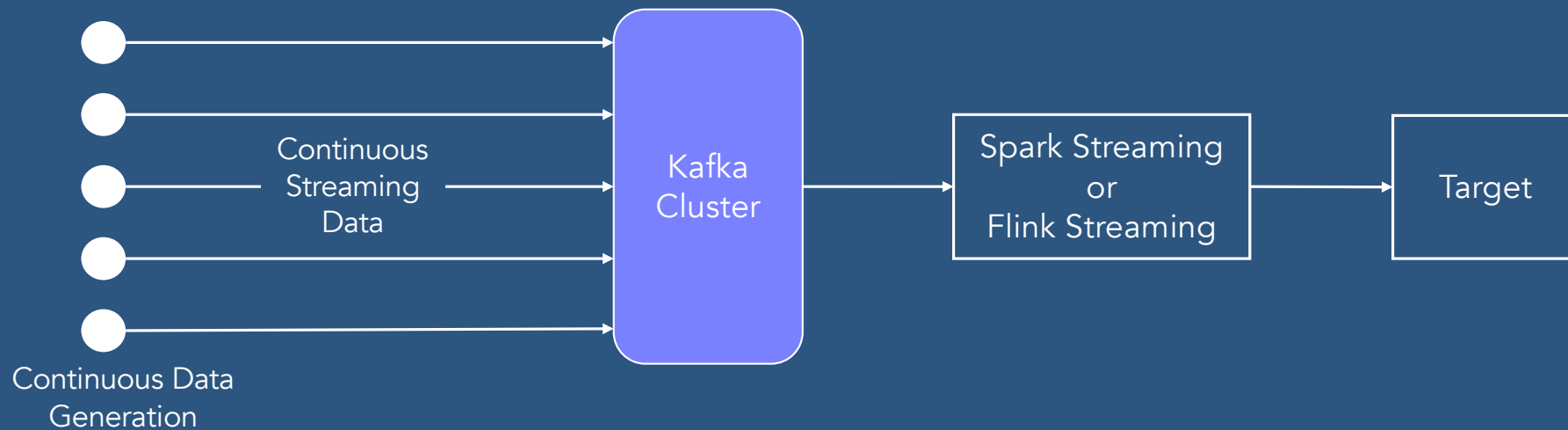
Kafka Introduction

Batch and Stream Processing



What is Kafka

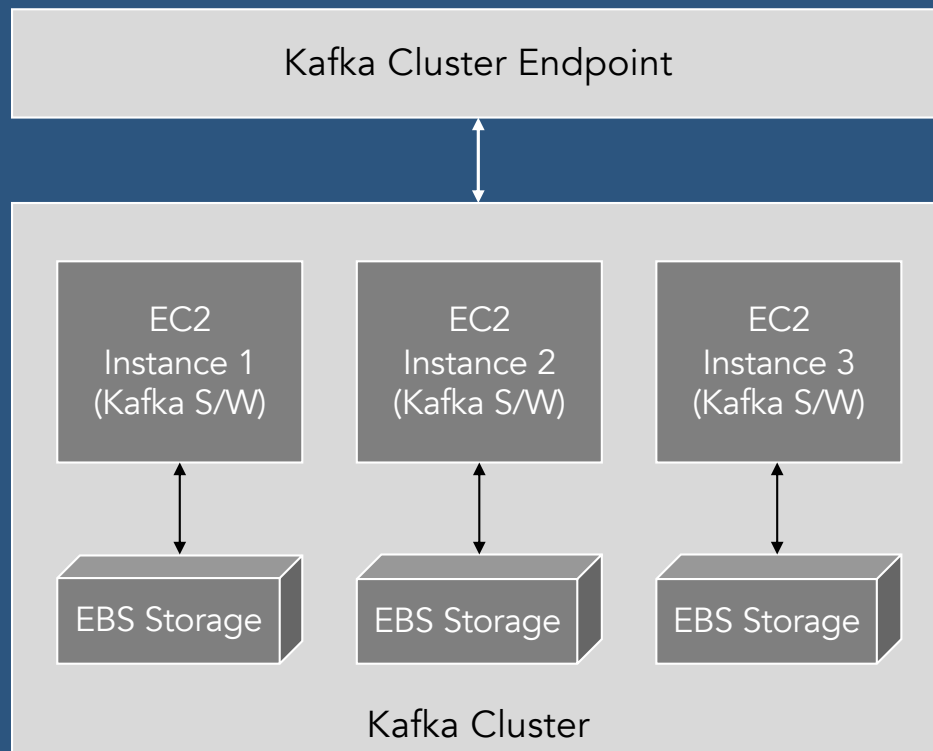
- Distributed streaming ingestion.
- Streaming data is ingested and stored durably for a determined duration.
- Why not use Spark to ingest data directly?



Streaming Sources

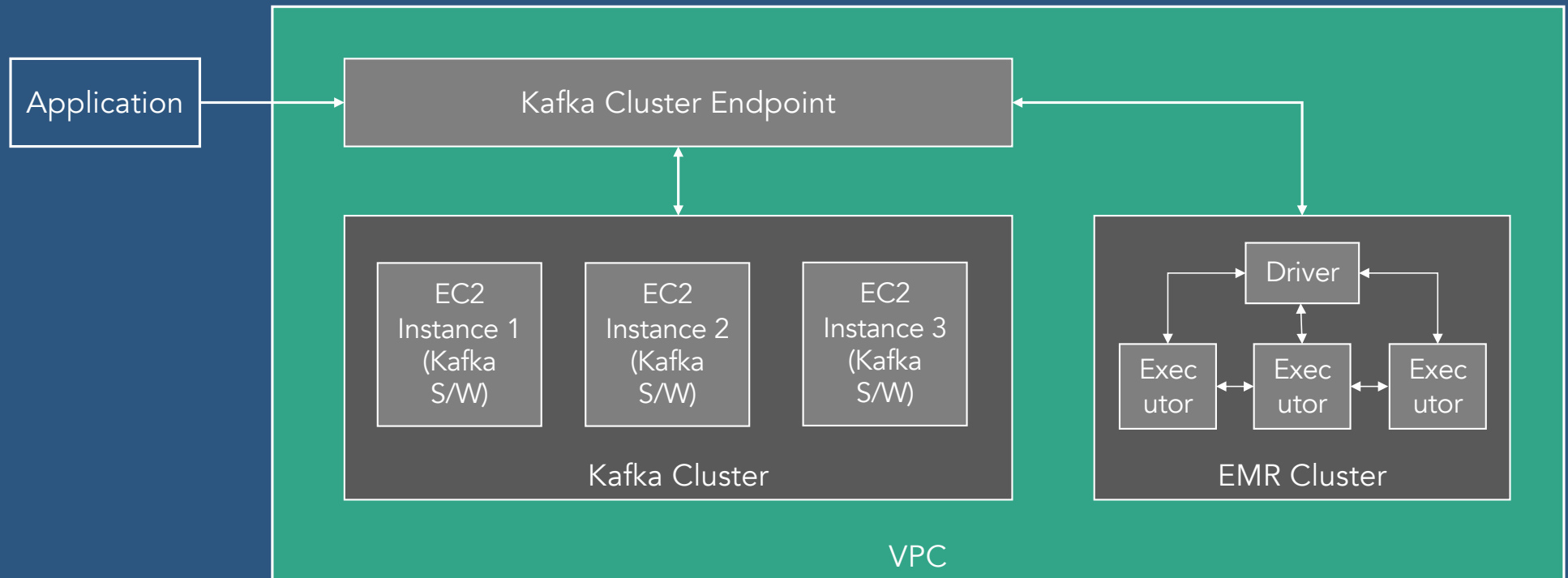
- Real-time applications
- IoT devices
- Transactions from UPI, Card, NetBanking etc.
- Change Data Capture (CDC) from SQL and NoSQL databases.

Kafka Cluster



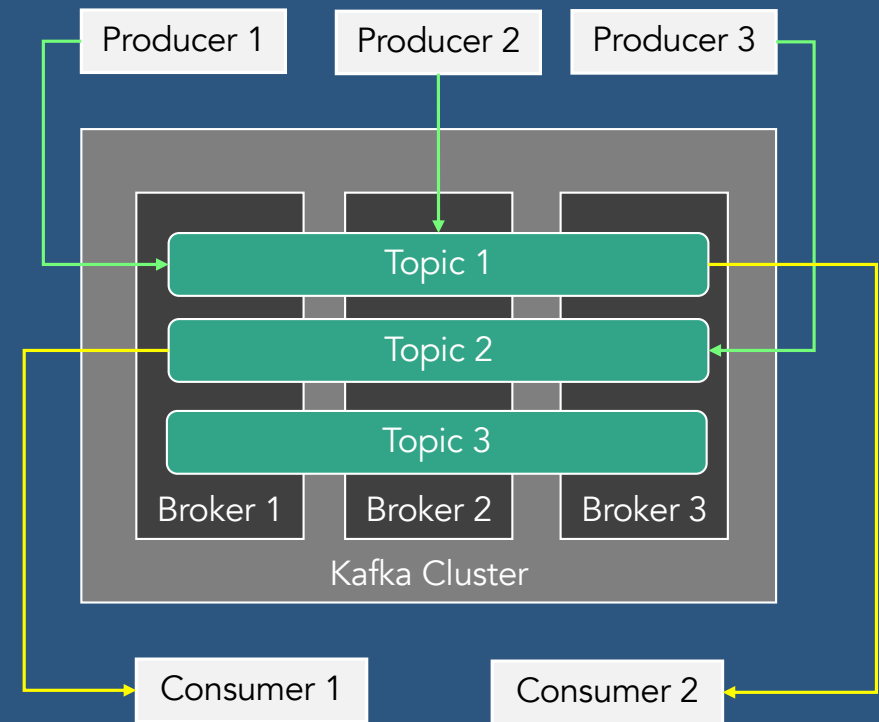
- Cluster is a collection of servers/ec2 instances.
- Each server is also known as Broker. Compute & Storage.
- Endpoint is used to connect to the cluster.
- Data is ingested and stored in the disk.
- Create Kafka cluster on AWS using MSK.

Kafka Cluster on AWS



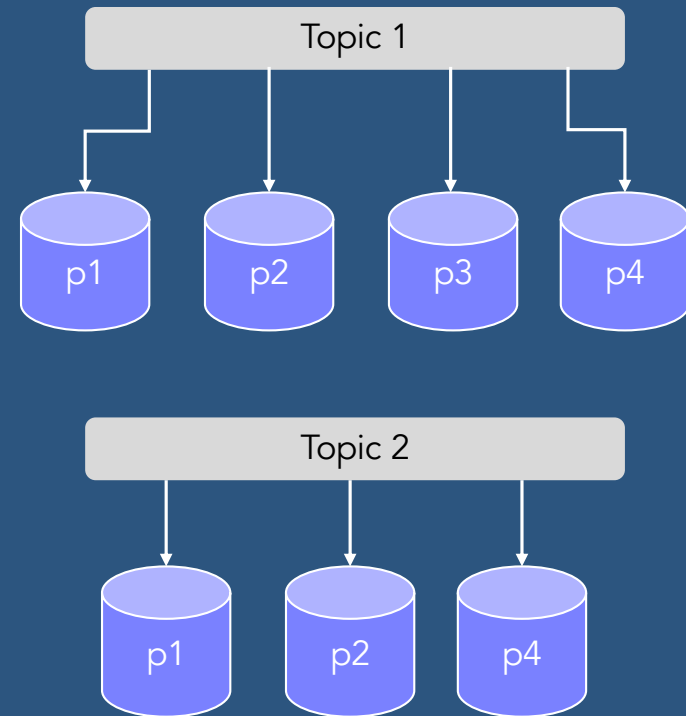
Kafka Components

- **Topics** – Where messages are sent and written.
- Topics are spread across the entire cluster.
- A single cluster can have multiple topics.



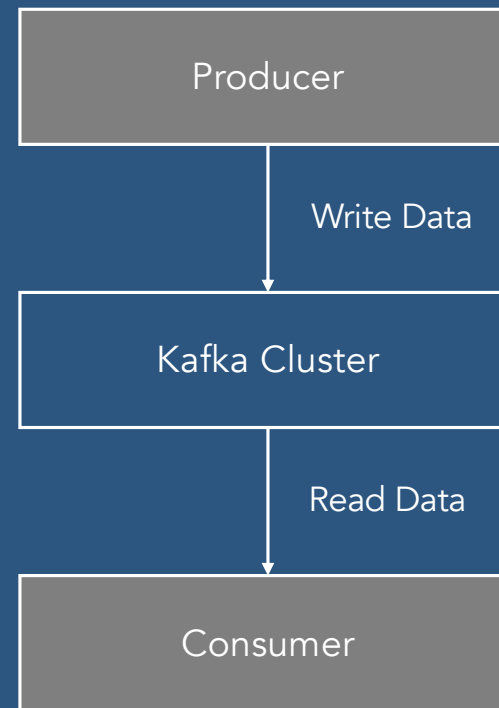
Kafka Components

- Topics – Where messages are sent and written.
- Topics are spread across the entire cluster.
- A single cluster can have multiple topics.
- **Partitions** – A single topic contains multiple partitions.
- Partition stores subset of the entire data or all messages that are sent to the topic.
- Let's create 2 topics.



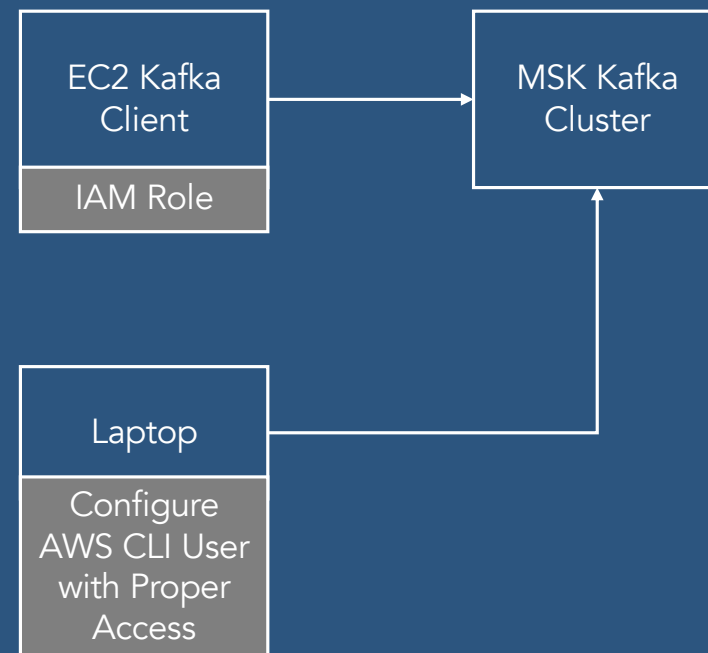
Producer & Consumer

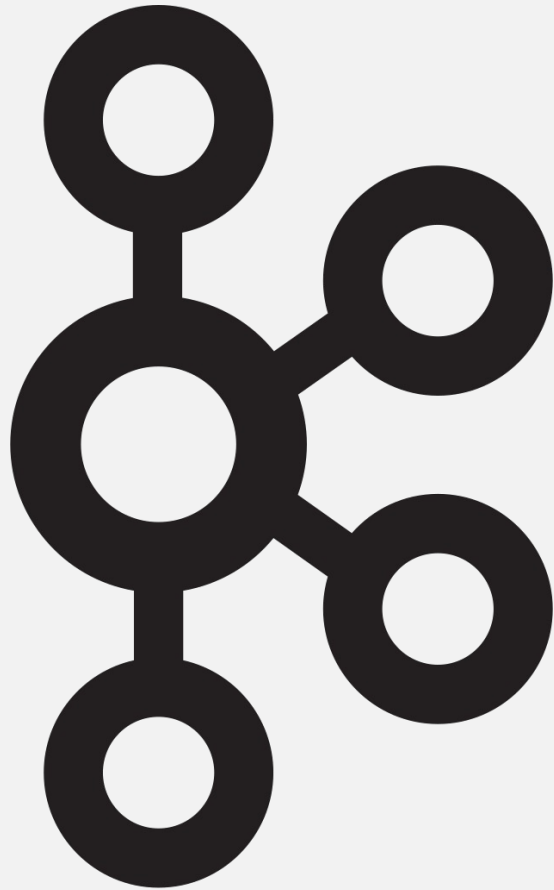
- Producer sends data to Kafka cluster.
- Consumer reads data from Kafka cluster.
- Both producer & consumer use Kafka SDK to write to the cluster and read from the cluster.
- Producer – Real-time Applications, CDC, IoT Devices.
- Consumer – Spark, Reader Applications.



Install Kafka CLI on EC2

- Download Java. **Kafka is built on JVM.**
- Download & extract Kafka. Set up environment variables.
- Authentication setup for MSK cluster using IAM role.
- Attach IAM role to EC2 instance.
- Add AWS IAM JAR in CLASSPATH.
- Create and set client configuration file.
- Use Kafka scripts to create, modify, list, describe topics.

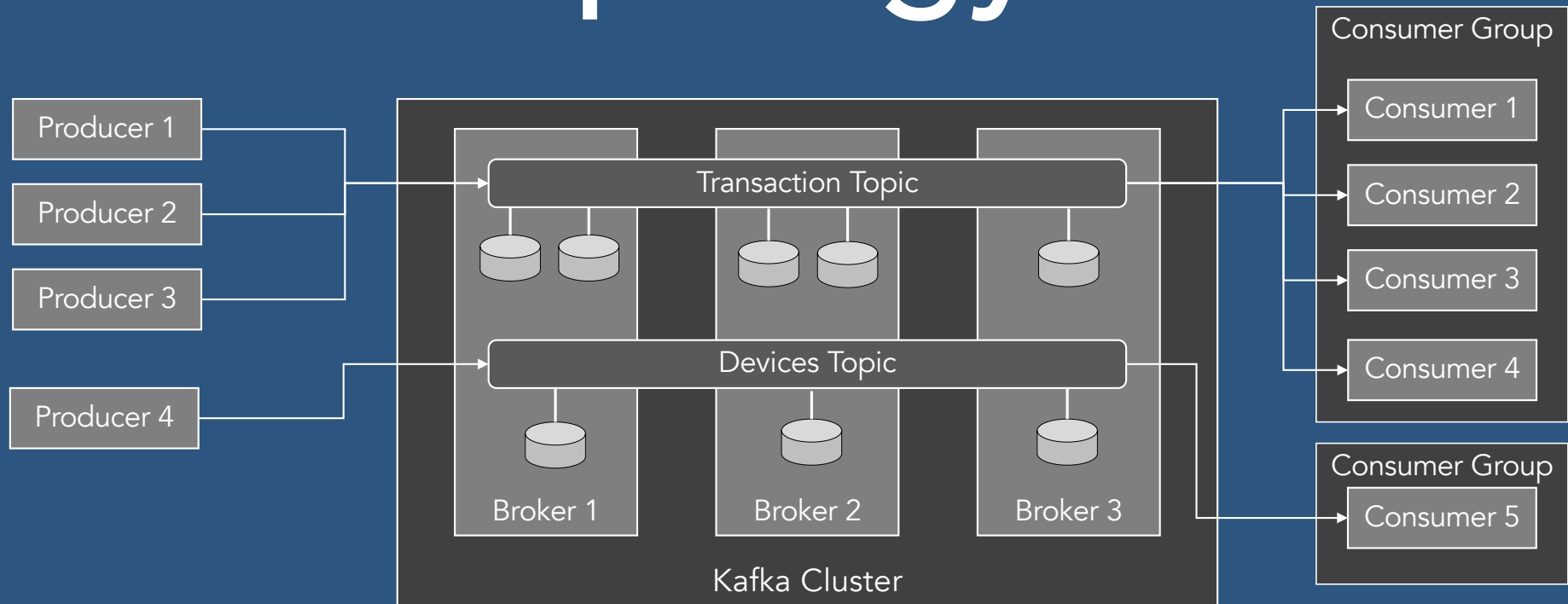




Knowing Kafka Up Close

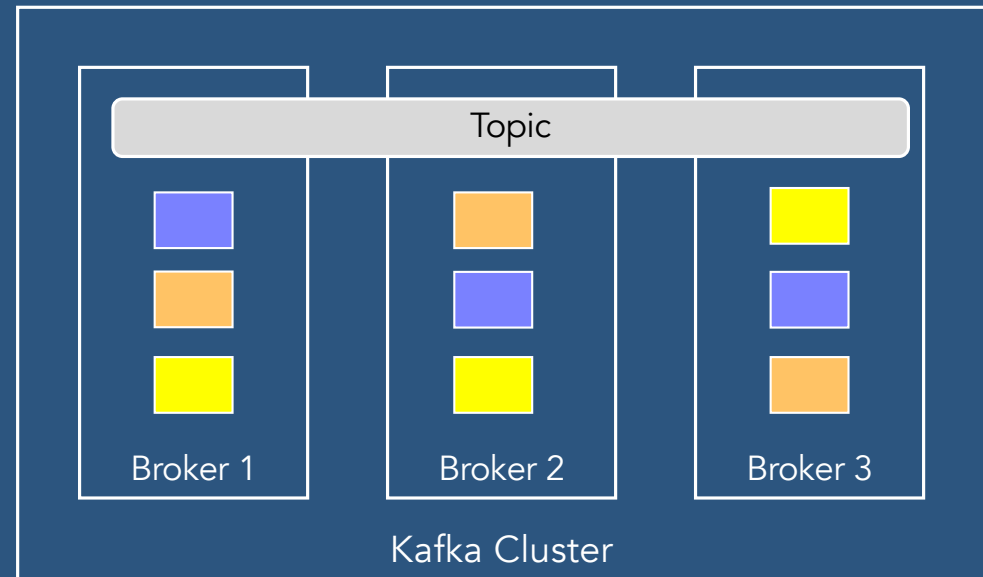
(Theory)

Kafka Topology



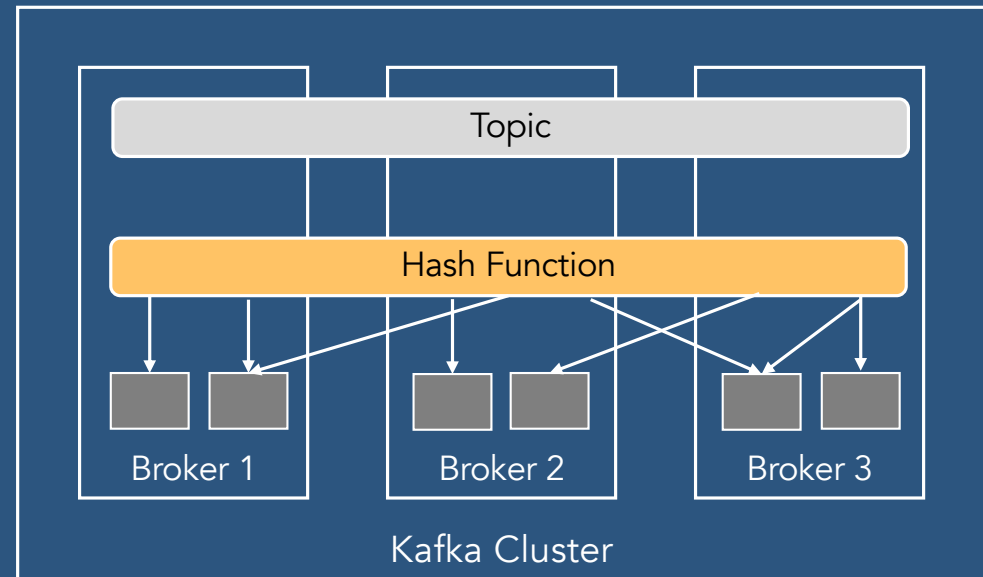
Topic Replication

- Single copy of the records are distributed across all brokers.
- Can be configured using Replication Factor.
- Automatically handled by Kafka.
- `--replication-factor 3`



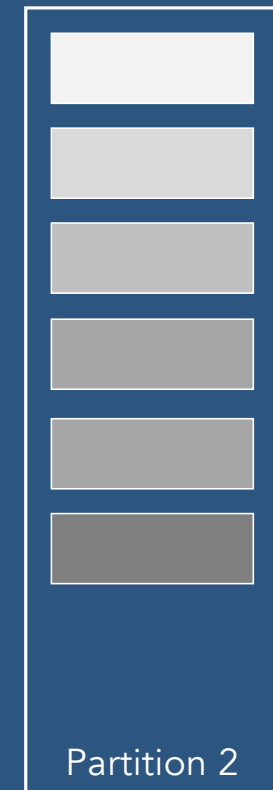
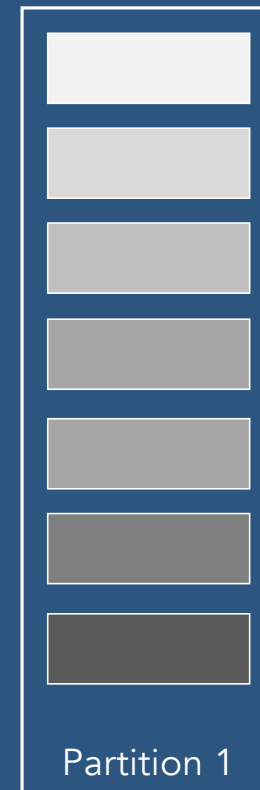
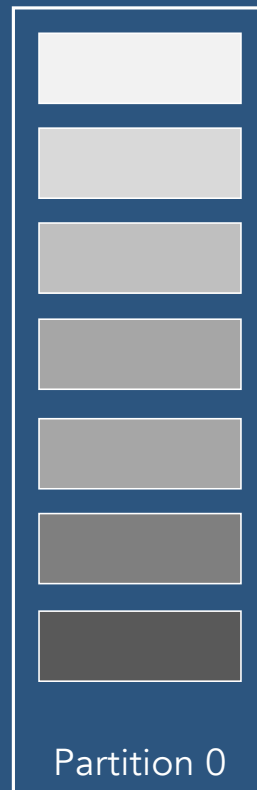
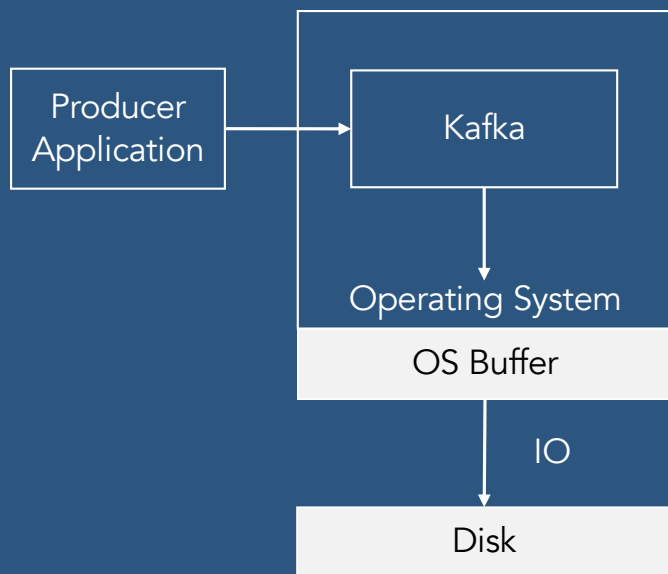
Message Distribution

- Message/record/data distribution across partitions.
- Message should have a key.
- Partition no. is decided on the hash value of the key. Key with same value will always land on the same partition.
- Ordering is maintained within a partition.
- What if there is no key – Messages are distributed in round robin fashion across all partitions. No ordering in a partition.

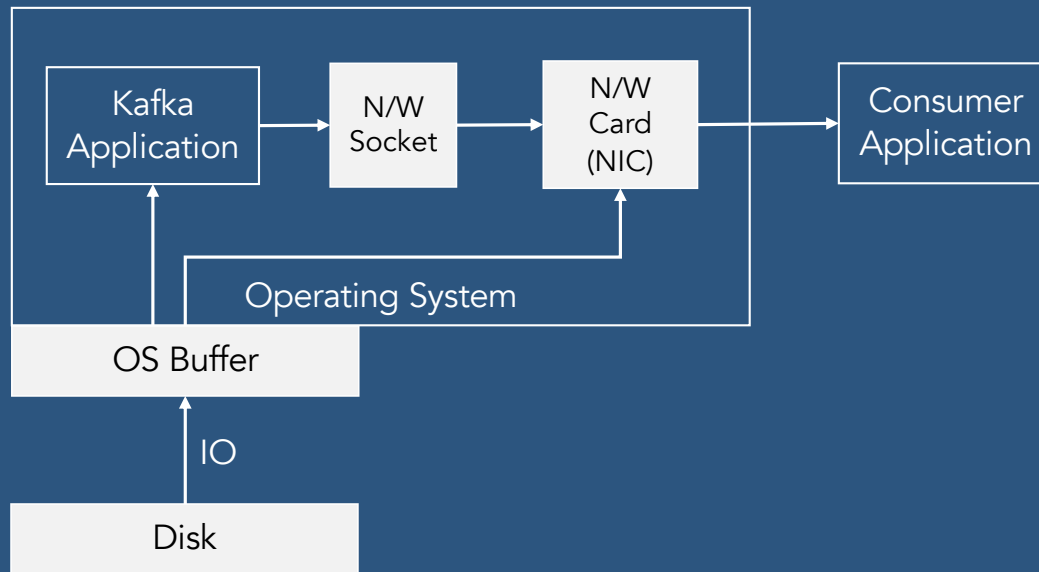


Write Data/Messages

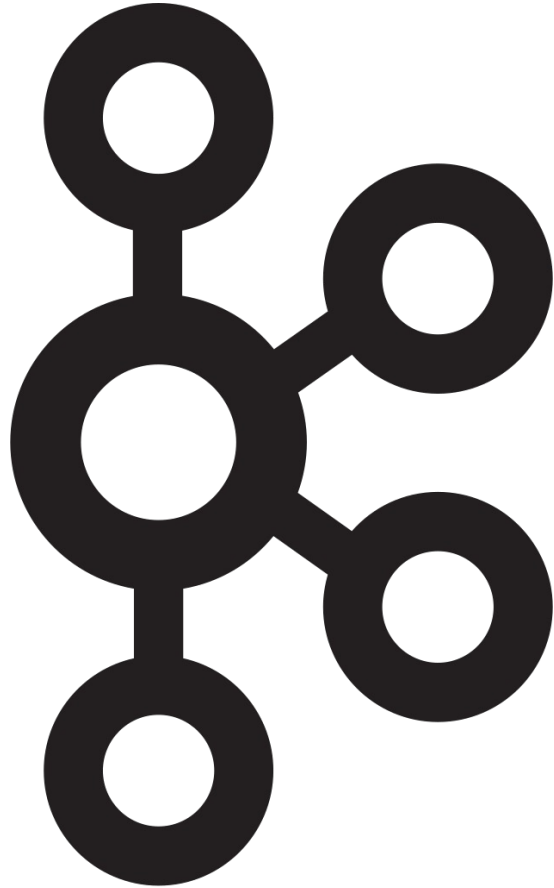
- Append Only Logs.
- Sequential Write.



Read Data/Messages



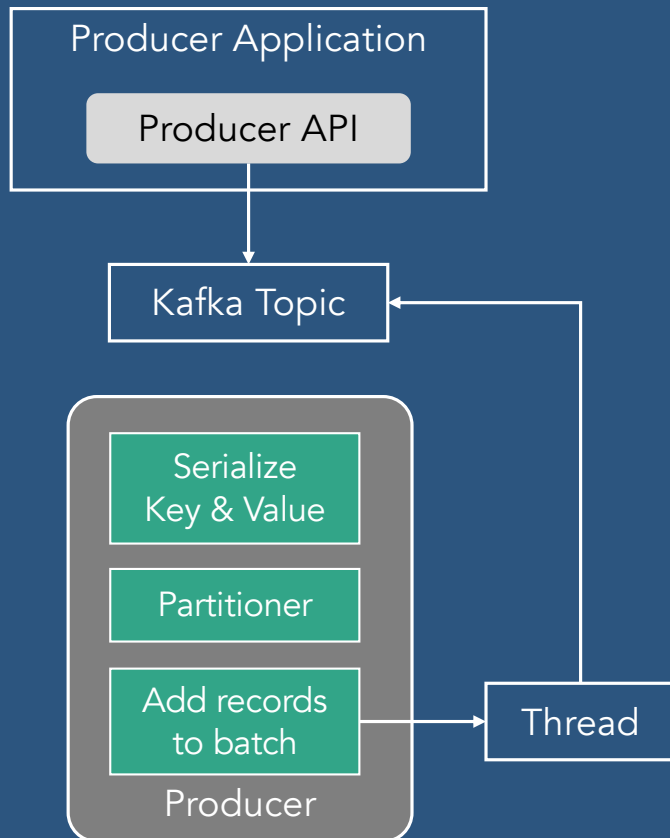
- How typically data is read and sent.
- What Kafka does differently.
- Direct Memory Access.



Producer & Consumer Applications

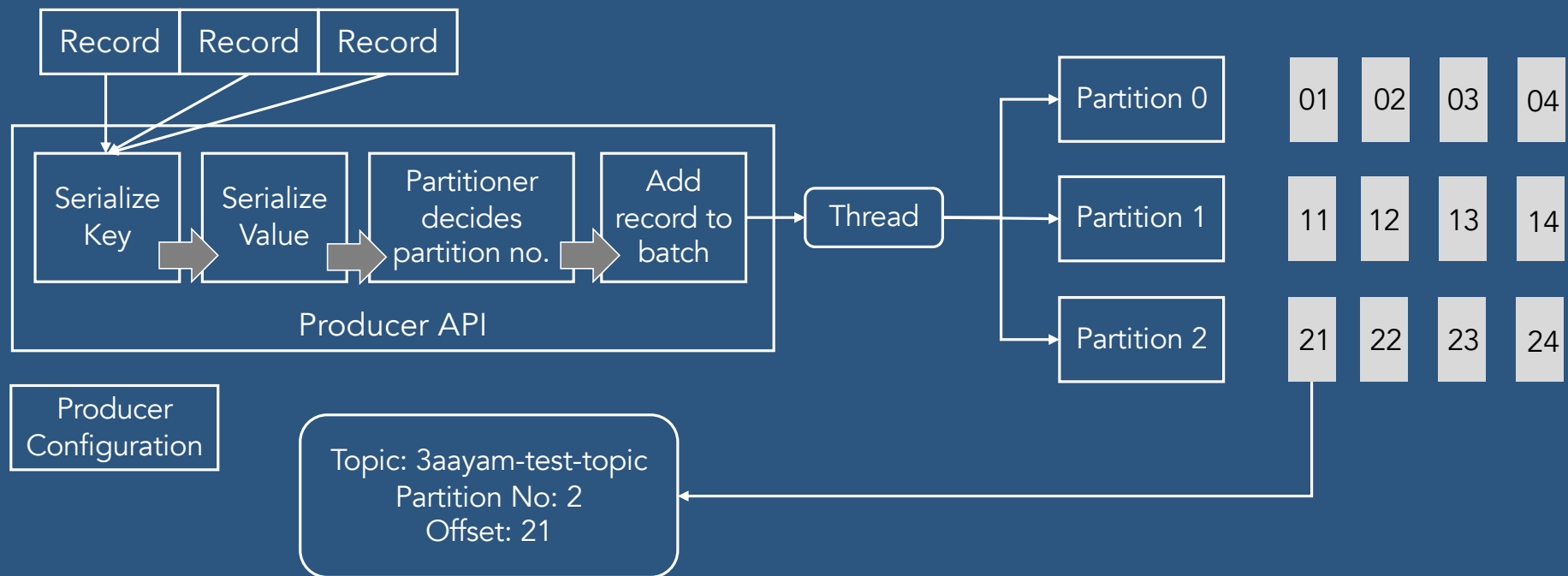
www.3aayaam.in

Producer Application



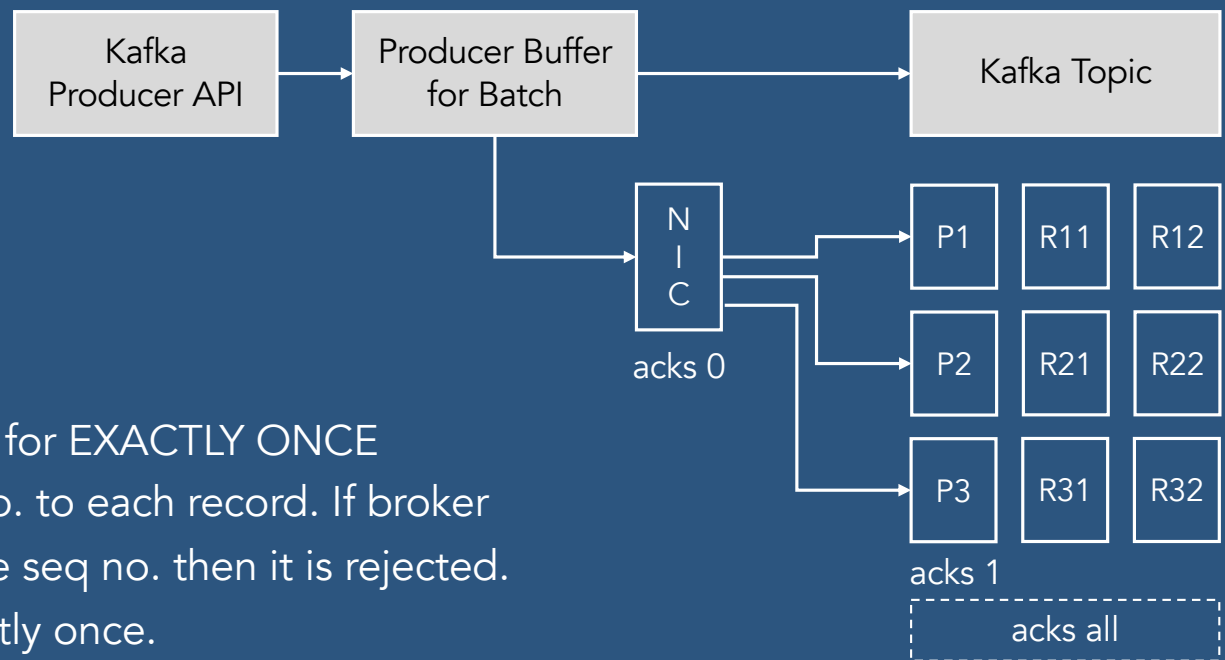
- Producer API is used to send messages to a Kafka topic.
- Python library – kafka-python. `pip3 install kafka-python`.
- We use Kafka Topic name, timestamp, record key and record value to send data to Kafka.
- Steps:
 - Serialize key and value.
 - Partitioner will choose partition no. based on hash value of key.
 - Record is added to a batch that would be sent to the same partition.
 - Separate thread sends those records to Kafka brokers.
 - Receives acknowledgement with topic, partition and offset of record in the partition.

Producer to Partitions



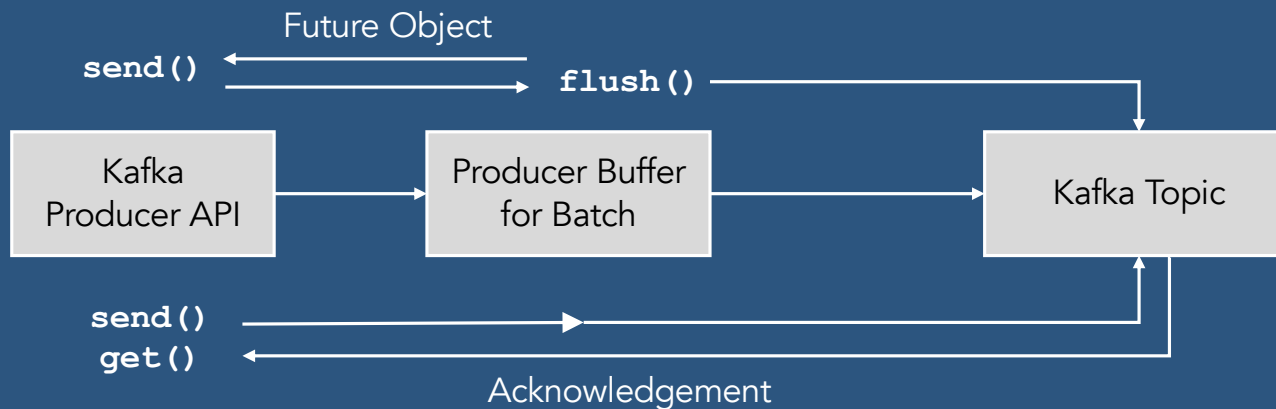
Producer Implementation

- from kafka import KafkaProducer
- Producer Configurations
 - acks (0, 1, all)
 - client_id
 - key_serializer
 - value_serializer
 - request_timeout_ms
 - batch_size
 - enable_idempotence – Used for EXACTLY ONCE semantics. Adds sequence no. to each record. If broker receives record with the same seq no. then it is rejected. Each record will be sent exactly once.



Sync & Async Send

- Producer Methods
 - `send` – Asynchronous.
 - `flush` – Buffered records are sent immediately.
 - `get` – For synchronous call.
 - `close` – Close producer.



kafka-python library

```
from kafka import KafkaProducer

producer = KafkaProducer(bootstrap_servers='<msk-endpoint>', value_serializer=<json>
                           enable_idempotence=True, acks=1, request_timeout_ms=2000)

record = {'topic': 'Kafka', 'name': '3aayaam'}
producer.send('3aayaam_test_topic', key=<key>, value=record)
response = producer.send('3aayaam_test_topic', key=<key>, value=record)
sync_response = response.get(timeout=5)
producer.flush()
producer.close()
```

HansOn - Producer

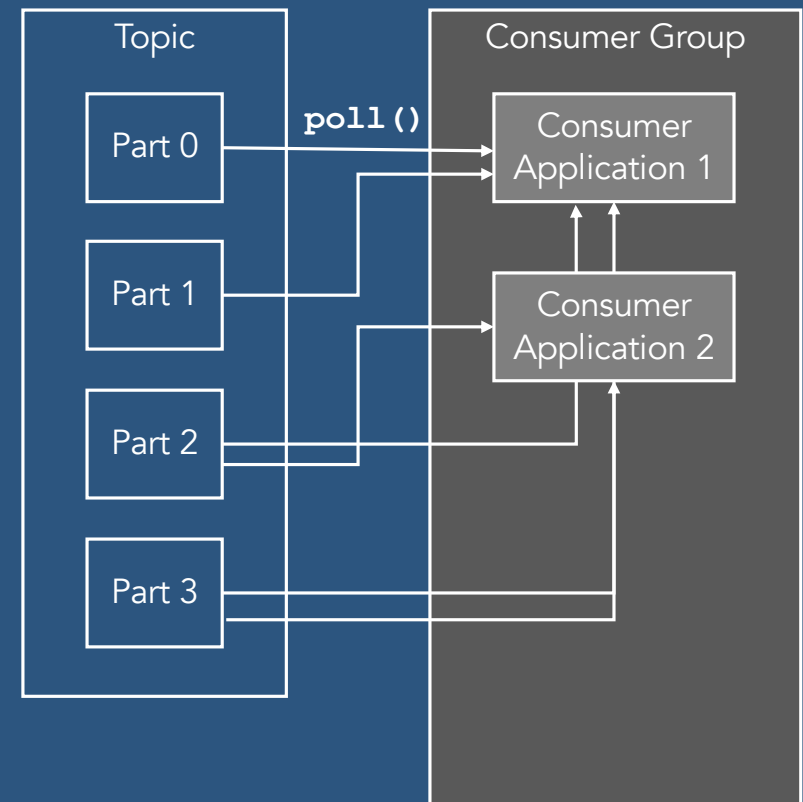
- Create a producer application using PyCharm and send to Kafka topic created in AWS MSK provisioned cluster.
- Read a file and send each record to Kafka topic one-by-one.

HansOn - Producer

- Create Kafka provisioned cluster.
- Enable "Public Access" from Network Settings.
- Install and configure Kafka Client on laptop.
- Install `kafka-python` and `aws-msk-iam-sasl-signer-python` libraries on laptop.
- Download SSL Certificate on laptop.
- Assign IAM policy ([3aayaam-msk-ec2-iam-policy](#)) to 3aayaam-admin/root user.
Configure AWS CLI with 3aayaam-admin/root user.

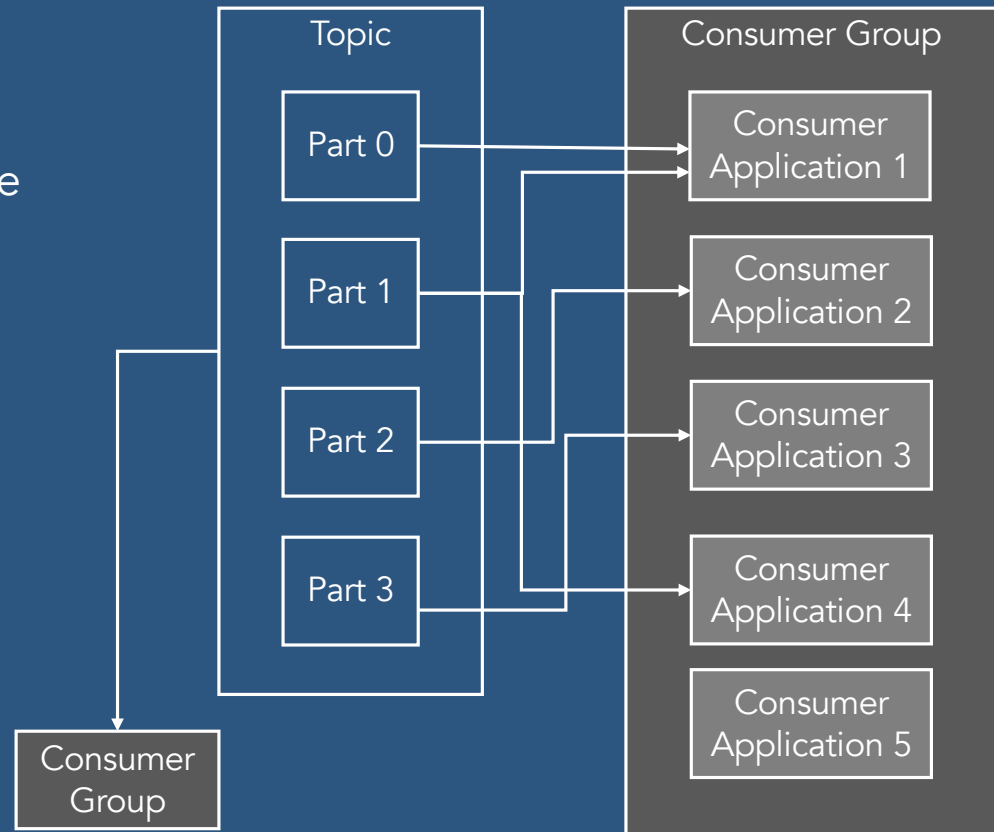
Consumer Application

- Consumer API is used to read messages from Kafka topic.
- Consumer Group – Collection of multiple consumers.
- Consumer has to subscribe to a topic.



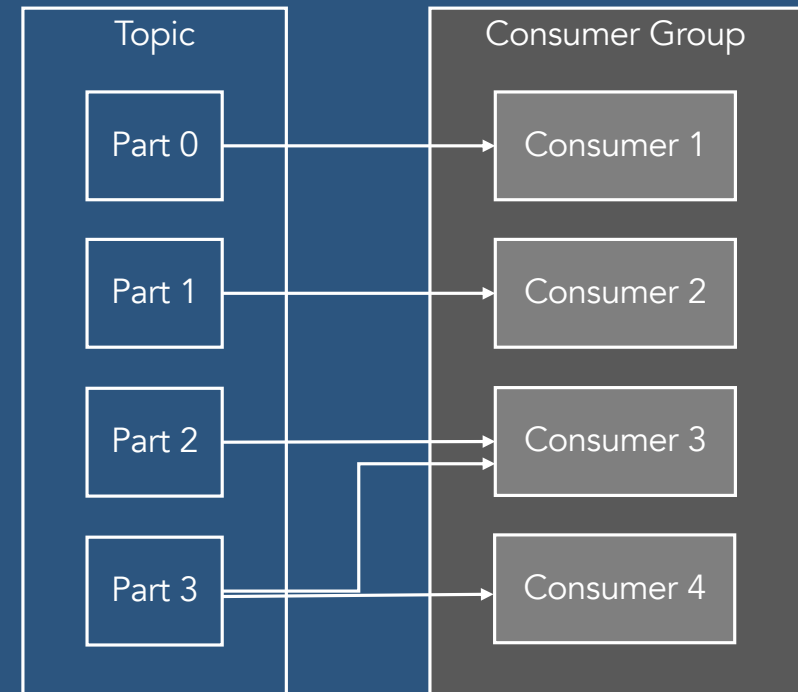
Consumer Application

- Consumer API is used to read messages from Kafka topic.
- Consumer Group – Collection of multiple consumers.
- Max no. of consumers in a Consumer group is equal to the no. of partitions of the topic that the consumer group is reading from.
- There can be one or more consumer groups reading from a topic.
- Adding consumers in a consumer group provides scaling without impacting performance.



Partition Rebalance

- Consumer ownership.
- Adding or removing consumers changes ownership of consumers. Partition Rebalance.
- Two types of Rebalance: Eager & Cooperative.
- Eager Rebalance Steps:
 - Stop all consumers and give up ownership.
 - Rejoin and get new ownership.
 - Incurs downtime.
- Cooperative Rebalance Steps:
 - Subset of consumers give up ownership.
 - New consumers own the unassigned partitions.
 - Lesser downtime.



Consumer Implementation

- from kafka import KafkaConsumer
- Consumer Configurations
 - Subscribe to a topic.
 - bootstrap_servers, client_id.
 - group_id – Name of consumer group.
 - key_deserializer & value_deserializer.
 - max_poll_records (500)
 - fetch_min_bytes (1 byte)
 - fetch_max_wait_ms (500)
 - request_timeout_ms
 - auto_offset_reset – earliest or latest.
 - enable_auto_commit – Offset will be committed periodically based on auto_commit_interval_ms.
 - partition_assignment_strategy – RangePartitionAssignor, RoundRobinPartitionAssignor, StickyAssignor and CooperativeStickyAssignor.

```
from kafka import KafkaConsumer

consumer = KafkaConsumer(
    <topic>,
    bootstrap_servers = '<mks-endpoint>',
    group_id = '<consumer-group-name>',
    value_deserializer = '<json.loads>',
    auto_offset_reset = 'latest',
    enable_auto_commit = 'True'
)
```

HandsOn – Consumer

- Create Kafka topic with 6 partitions.
- Use a consumer group with one application to read data from Kafka Topic.
- Keep on adding consumers to the consumer group (using group_id) until the no. of consumers reach 8.
- Look at the behavior after adding each consumer.

Polling Data

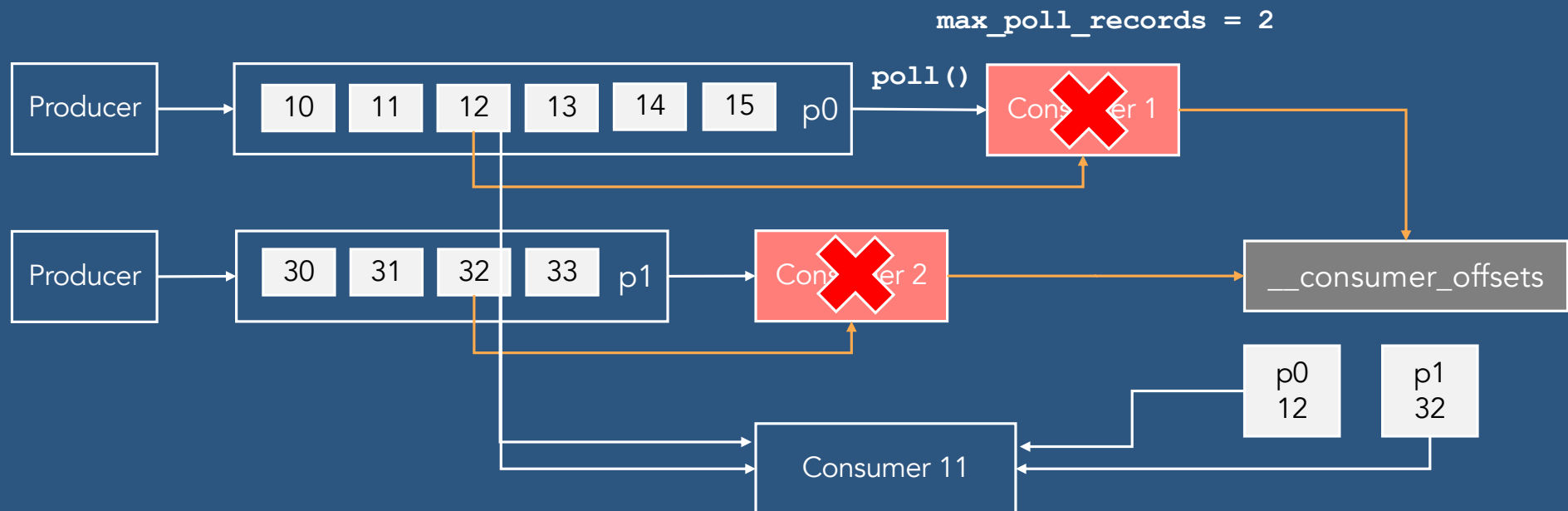
- `poll()` API. Data is fetched in batches.
- Kafka keeps a note of the records fetched in each poll using Offset.
- Handled automatically by Consumer object when used with "for" loop.
- The for loop uses poll API underneath. This should be used as best practice.

```
from kafka import KafkaConsumer

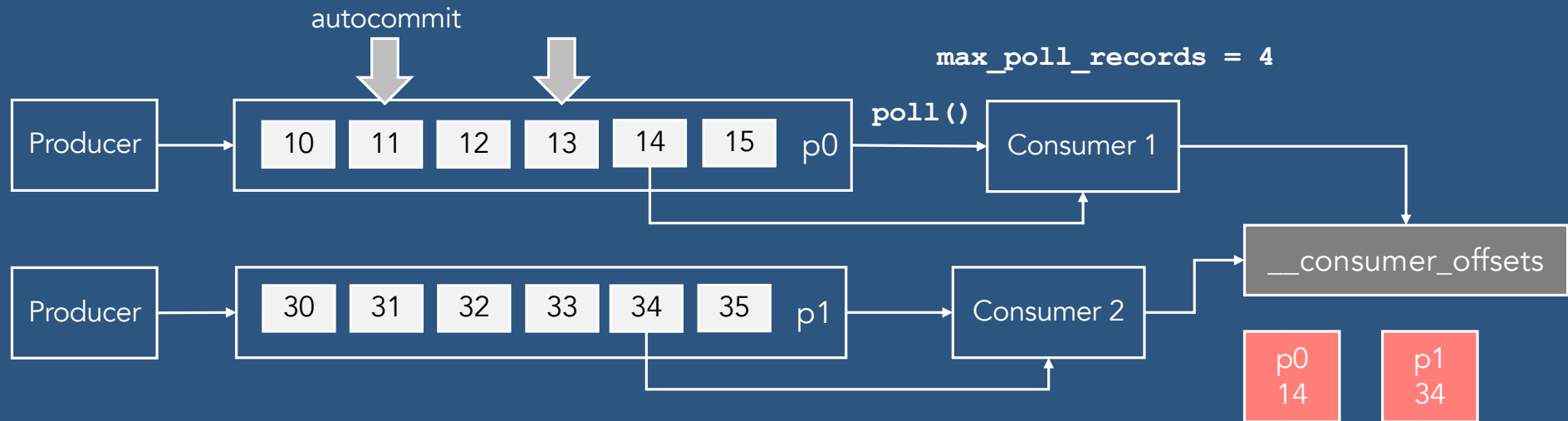
consumer = KafkaConsumer(...)

while True:
    record = consumer.poll(timeout_ms=5000, max_records=<n>)
```

Offset



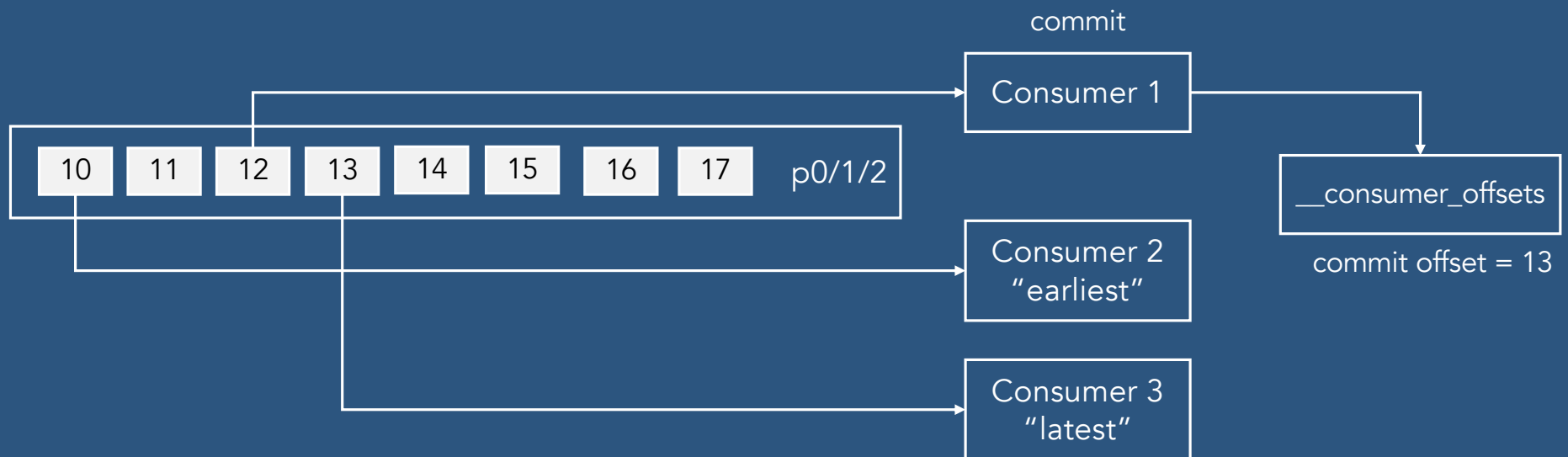
Commit Offset



- `enable_auto_commit = True` (every 5 seconds). `auto_commit_interval_ms` (5000)
- Any problem??
- `consumer.commit()` – Application will wait until broker responds to the commit request.
- `consumer.commit_async()` – Application will NOT wait for broker response.

Auto Offset Reset

- `auto_offset_reset` – Earliest & Latest.



Offset Management

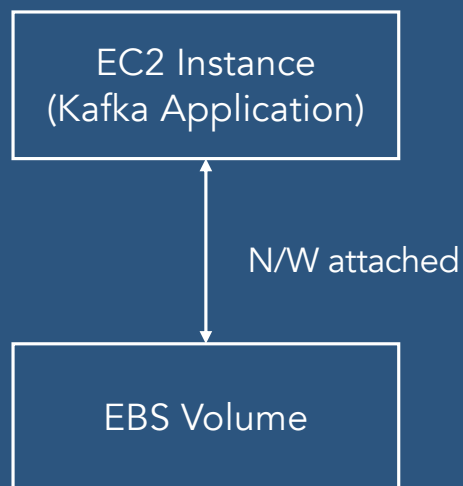
- `assign()`, `assignment()`
- `partitions_for_topic(<topic>)`
- `seek_to_beginning`
- `seek_to_end()`
- `seek(<partition>, <offset>)`
- `pause()` & `resume()`
- `offsets_for_times(<timestamp>)`
- `metrics()`
- `subscribe()`

Cluster Metadata

- `partitions_for_topic()`
- `partitions_for_broker(<broker-id>)`
- `available_partitions_for_topic(<topic-name>)`
- `broker_metadata()`

AWS MSK Throughput

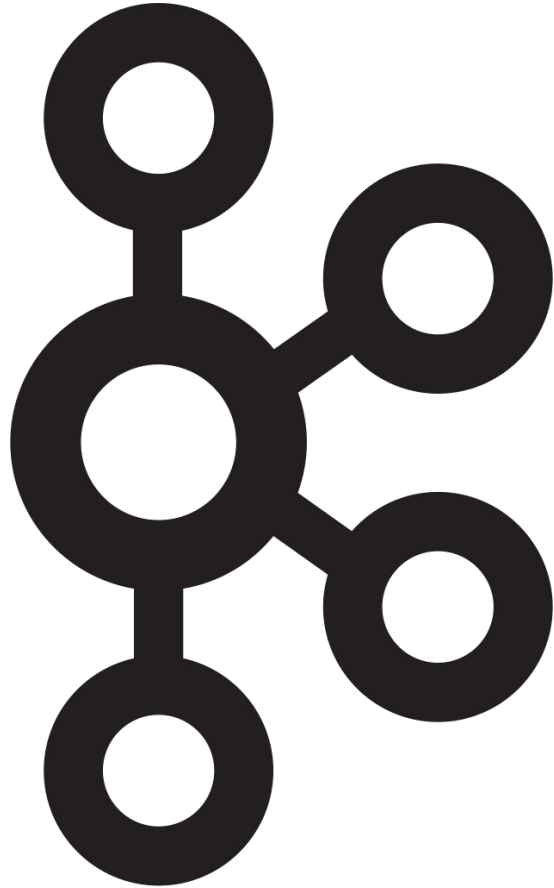
- MSK ingress (data-in) and egress (data-out) throughput.
- Depends on EC2 to EBS network.
- For EBS volume size more than 10 GB 250 MB/s can be provisioned.



broker size	Maximum storage throughput (MiB/second)
kafka.m5.4xlarge	593
kafka.m5.8xlarge	850
kafka.m5.12xlarge	1000
kafka.m5.16xlarge	1000
kafka.m5.24xlarge	1000
kafka.m7g.2xlarge	312.5
kafka.m7g.4xlarge	625
kafka.m7g.8xlarge	1000
kafka.m7g.12xlarge	1000
kafka.m7g.16xlarge	1000

What should be the no. of Partitions in a Topic?

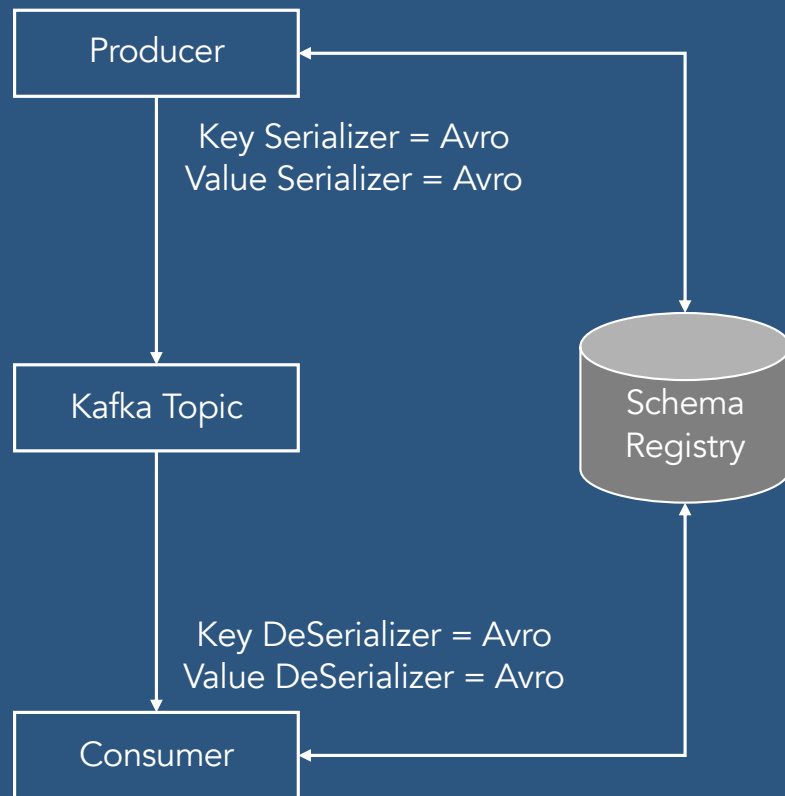
- MSK ingress (data-in) and egress (data-out) throughput should be considered while calculating partitions.
- Should be planned during topic creation.
- Factors in deciding no. of partitions: i) Ingestion volume. ii) Consumption volume. iii) No. of consumers.
- Example: All IoT devices/RTAs generating 1,000,000 messages in every 5 seconds. There are various Spark applications running that read data from the topic with a rate of 2,000,000 messages per second. Each payload is of size 1KB. What should be the no. of partitions of the topic?
- WarpStream – Diskless, Auto-scaled, Kafka Compatible.



Schema Registry

www.3aayaam.in

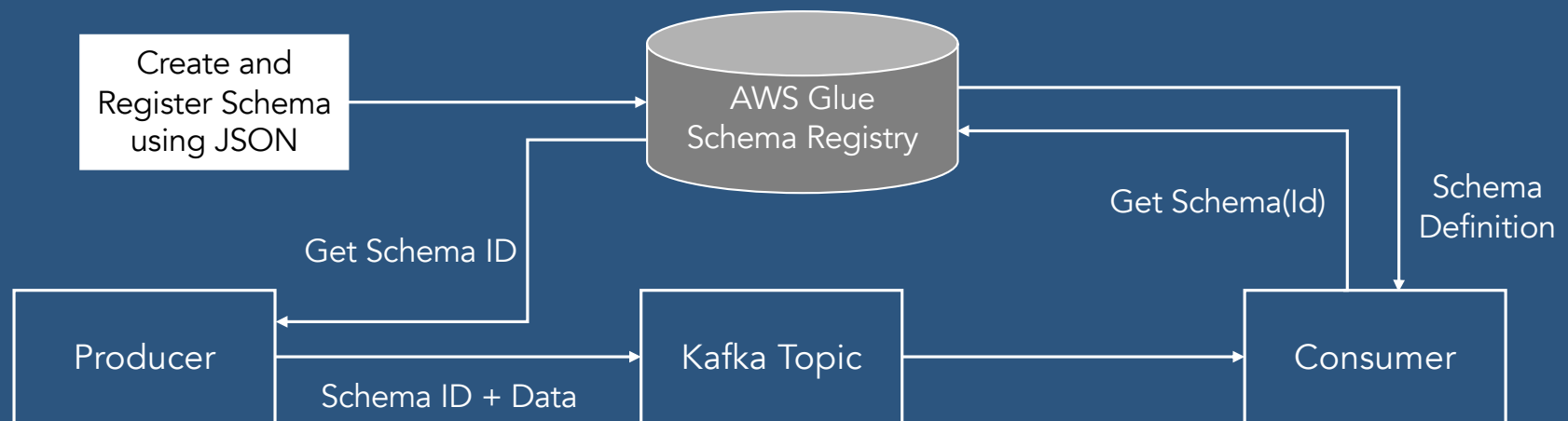
Avro Serializer & De-serializer



- Producer can send messages using Avro format.
- Key & Value serializer should be Avro.
- Consumer has to read using using Avro deserializers.
- Schema has to be sent with every record
- Use schema registry to store schema.
- Glue Schema Registry
 - AVRO
 - JSON
 - ProtoBuf

Implementation

- Create schema in Glue Schema Registry. Using Python (with .avsc file) or Manually.
- Schema name should be same as Topic name. Define schema using JSON.
- Producer provides schema along with data, which is validated with schema registry.
- Consumer gets schema details from registry.



Avro Data Types

- boolean
- int
- long
- float
- double
- bytes
- string

Python Code

- `pip3 install "aws-glue-schema-registry[kafka-python]"`
- `from kafka import KafkaProducer`
- `from aws_schema_registry import SchemaRegistryClient`
- `from aws_schema_registry.avro import AvroSchema`
- `from aws_schema_registry.adapter.kafka import KafkaSerializer`
- Have to use `get_schema_version` to fetch schema details.
- `producer.send(topic_name, value=(data, schema))`

Steps

- Create Kafka topic.
- Create Glue schema registry.
- Create Glue schema with the same name as the topic.
- Write Python code to produce data and consume data using Avro serializer and deserializer.

Schema Evolution

- Schema changes (evolves) over time – Add or drop columns, change column data types.
- Schema Versions – Schema modification creates a new version. A single schema can have multiple versions.
- Schema Version Checkpoint – Schema baseline. Compatibility is always checked against checkpointed version. Can be modified.
- Compatibility – Whether schema can evolve over time. Formats – AVRO, JSON & Protobuf.
- Fields or Columns types – Required (R) & Optional (O).
- Data Contract between producer & consumer.

Schema Compatibility (Avro)

- DISABLED – Compatibility is not allowed.
- NONE – To checks are applied, new schema version is accepted.
- BACKWARD – Consumers are allowed to read current and previous version when columns are added or removed.
 - Drop required field – Success. Extra field in old records will be ignored by consumer.
 - Add required field – Fail. Consumers will not be able to read old records as required field was not there.
 - Add optional field – Fail.
- BACKWARD_ALL – Consumers are allowed to read current and all previous versions.



Consumer

Schema Compatibility (Avro)

- FORWARD – Allows consumers to read current and only next version.
 - Add required field – Success.
 - Drop required field – Fail.
 - Drop optional field – Success.
- FORWARD_ALL – Allows consumers to read current and all new versions.
- FULL – Consumers are allowed to read current, previous one and next one version.
- FULL_ALL - Consumers are allowed to read current, all previous and all next versions.

Kafka Configuration

- Kafka Lag - How far behind a consumer is in reading the available messages. Depending on the consumer's purpose, it may need to read and process messages at a greater rate. A consumer group provides for stateless horizontal scalability.
- How far behind a follower partition is in replicating the messages from the leader partition. If a follower gets too far behind then it becomes an out-of-sync replica.
- Topic Configuration: (`--config <parm>=<value>`)
 - `max.message.bytes (topic) | message.max.bytes (broker)` – Max record size.
 - `flush.messages | flush.ms` – Flush from buffer to log file after these many messages or time.
 - `retention.bytes` – Max size of partition after which old records will be deleted.
 - `retention.ms` – Max retention period in millisecond.
 - `cleanup.policy` – delete or compact. What to do with files after retention period is reached, either delete or compact.
 - `delete.retention.ms` – Time after which compacted files will be deleted.

HandsOn – Schema Evolution

- Create/Register schema with required and optional fields. Use BACKWARD compatibility mode.
- Register new schema for different use cases.

- Kafka Cluster
- Broker, Topic
- Producer, Consumer
- Kafka Client
- Replication, Message Distribution
- Write & Read Data
- Producer & Consumer API, Configuration
- Partitions, Sync & Async send
- Offset Management
- Kafka Throughput
- Schema Registry

Summary

Thank You.
SAVE ANIMALS!!! Save Earth

